Problem A. Asteroid Mining

All masses are totally ordered by divisibility. Add dummy chunks of every integer mass with value 0, so we may treat the chosen total mass as exactly M. The solution repeatedly reduces the smallest mass scale.

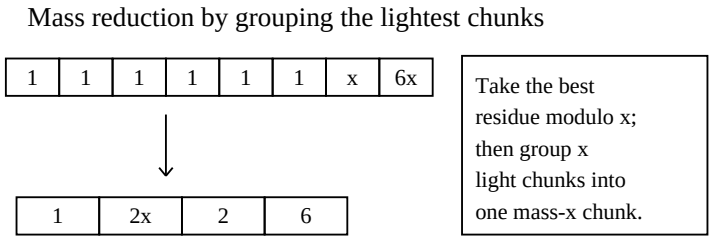

Figure 1. Grouping the lightest chunks turns them into the next mass scale.

Small M subtasks – knapsack baseline

When M is small, a standard $0/1$ knapsack over mass up to M works: $dp[t]$ is the best value with total mass t. This covers the early marks with small M and moderate N.

Equal masses subtask

If all masses are equal to m, take $\lfloor M/m \rfloor$ chunks of highest value. This is exactly the last stage of the full greedy reduction when no next mass scale exists.

At most two distinct masses

Normalize by the smaller mass. Suppose the two lowest masses are 1 and x. The number of mass-1 chunks used is fixed modulo x. Take the best M mod x such chunks first, then group the remaining mass-1 chunks into batches of x in descending value order. Each batch becomes one synthetic chunk of mass x.

Full solution

Repeat the two-mass reduction. If the current minimum mass is $d > 1$, divide all masses and M by d, replacing M by $\lfloor M/d \rfloor$ when necessary. Then let x be the next distinct mass after normalization. Choose the required residue of mass-1 chunks, group the rest into batches of x, divide by x, and continue. Each iteration removes one mass scale or reduces M substantially, so there are $O(\log M)$ rounds. Sorting or priority queues over chunks give $O(N \log N \log M)$ overall.

Problem D. Restaurant Recommendation Rescue

The recommendation process is a rooted tree in BFS order. If a vertex has child count c_i, define $a_i = c_i - 1$. Valid arrays are exactly Raney sequences: $a_i \geq -1$, every proper prefix sum is nonnegative, and the total sum is -1. The task is to find cyclic shifts of B that are valid, and to maintain the answer under swaps.

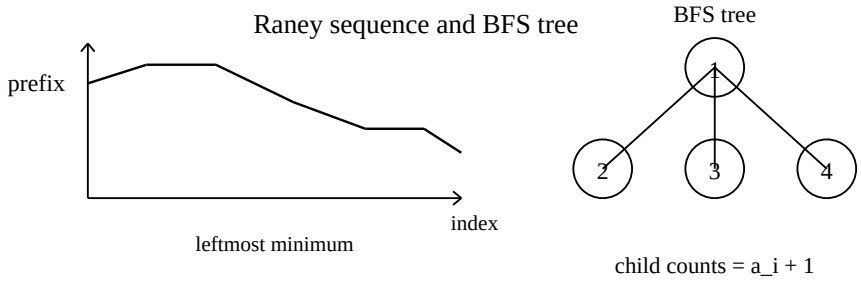

Figure 2. Raney prefix sums correspond to BFS child counts in a rooted tree.

Subtask 1 – brute force shifts

For $N \leq 8$ and no updates, try every cyclic shift and explicitly check whether it can be a valid BFS child-count sequence.

Subtask 2 – $O(N^2)$ Raney checks

Use the Raney sequence characterization. For every cyclic shift, scan its prefix sums and check the three conditions. Equivalently, run the constructive tree-building algorithm for each shift.

Subtask 3 – unique valid rotation

For any array a with $a_i \geq -1$ and total sum -1, exactly one cyclic shift is a Raney sequence. Let s_i be prefix sums. The valid shift starts just after the leftmost minimum prefix sum. With no updates, find that minimum prefix in $O(N)$.

Subtask 4 – swaps with a segment tree

A segment tree node for interval $[l, r]$ stores the total sum, the minimum prefix sum inside the interval, and the smallest index attaining that minimum. The merge rule is the usual prefix-min merge. Each swap changes two array values, so it becomes two point updates. The root gives the current leftmost minimum prefix, hence the unique valid shift if the global sum is -1 and all values are at least -1.

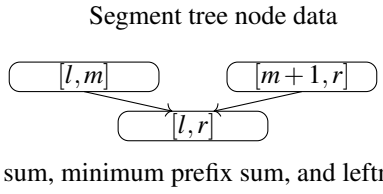

Figure 3. Prefix-min segment tree used for the dynamic rotation.

Problem E. Patrol Robot

We must draw a non-crossing segment graph so that the deterministic right-turn walk visits all vertices indefinitely from any starting directed edge. The construction is geometric and recursive.

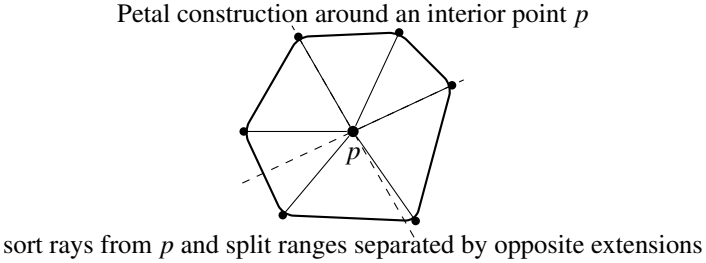

Figure 4. Interior point p partitions the hull vertices into petals.

Subtask 1 – $N \leq 4$ by casework

For $N = 2$, draw the only segment. For $N = 3$, draw the triangle. For $N = 4$, draw the convex quadrilateral if all four points are on the hull; otherwise, connect the interior point to the three triangle vertices. A counterclockwise orientation predicate distinguishes the cases.

Subtask 2 – backtracking for $N \leq 8$

Try subsets of possible segments recursively, pruning as soon as two segments intersect. The naive upper bound is $2^{N(N-1)/2}$, but non-crossing pruning makes the search small enough for $N \leq 8$.

Subtask 3 – all points convex

Draw the edges of the convex polygon. The robot then follows the polygon cycle forever. The polygon can be found by sorting by angle or by a convex hull algorithm.

Subtask 4 – one interior point

Let p be the interior point. Consider all rays from p to hull vertices and their extensions through p. Sort the rays by angle. Consecutive groups not separated by an opposite extension form petals. Draw each petal so that the robot traverses an entire petal before switching to another petal, and always enters a petal at its leftmost segment from p.

Subtask 5 – full recursive construction

If the point set has two points, draw the edge. If all points are on the convex hull, draw the hull cycle. Otherwise pick an interior point p, partition the other points into petal groups, and recursively solve each $S_i \cup \{p\}$. There are at most N recursive calls. Each call may compute a hull and angle order in $O(N \log N)$, so the official bound is $O(N^2 \log N)$. The resulting graph is a cactus and uses at most $3(N-1)/2$ edges.

Problem F. Shopping Deals

Each deal chooses one quadrant relative to its point. We can either buy an item individually or cover it by selected deal quadrants. The solution optimizes the maximum savings from deals and subtracts from the sum of all item prices.

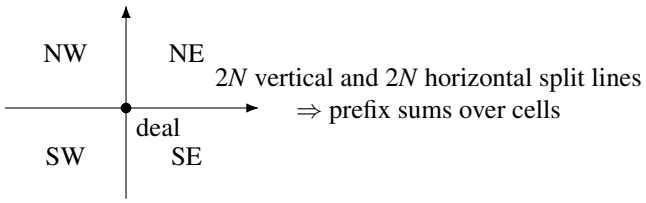

Figure 5. A deal chooses one quadrant; compression turns item sums into rectangle queries.

Subtask 1 – brute force deal assignments

For every deal, choose one of four quadrants or leave it unused. Test all 5^N assignments and compute the resulting cost by scanning items. The complexity is $O(NM \cdot 5^N)$.

Subtask 2 – bitmask DP over items

When M is small, process deals one by one and keep a bitmask of covered items. Each deal can cover one of four masks, or be skipped. This gives $O(N2^M)$.

Subtask 3 – at most four deals matter in the small setting

A key observation is that four chosen deals can always be oriented to cover the whole plane. Thus compare the four cheapest deals with every way to use at most three deals. For each choice, scan all items to compute coverage. The complexity is $O(N^3 M)$.

Subtasks 4 and 5 – coordinate compression and 2D prefix sums

The N deal points induce $2N$ vertical and $2N$ horizontal split lines. They divide the plane into $(2N+1)^2$ cells, and every deal region is a union of cells. Put item-price sums into these cells and build a 2D prefix-sum array. Now every union from at most three deals can be evaluated in $O(1)$ by inclusion-exclusion, reducing the previous approach to $O(M \log N + N^3)$.

Subtasks 6 and 7 – $O(N^2)$ case analysis for three deals

First handle using $0, 1, 2$, or 4 deals. For exactly three deals, all essential arrangements fall into four cases up to rotation and reflection: two regions are disjoint; one opposite quadrant is completely uncovered; only two opposite corners are covered; or two deals cover each other and the third covers a remaining corner. For each case, try one anchor deal or a pair of anchor deals and maintain prefix-optimum-like values so all choices are considered in $O(N^2)$. Some cases need second or third best choices to ensure the selected deals are distinct.

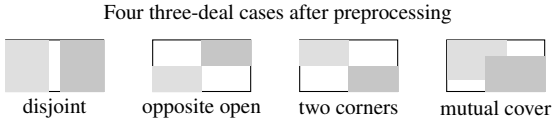

Figure 6. The four essential cases for exactly three deals.